Day 10 — Enterprise GenAI system design and Staff-level synthesis

Outcome

Be able to lead a system-design interview from ambiguous business problem to measurable architecture, explain reliability/security/AI quality, and present ownership and trade-offs at Senior or Staff level.

1. Interview design sequence

```
requirements
→ non-functional requirements
→ capacity
→ APIs
→ data model/access patterns
→ high-level architecture
→ sync/async paths
→ reliability
→ security/governance
→ observability/evaluation
→ cost
→ trade-offs/evolution
```

Do not begin with a fashionable framework. First establish the constraint that requires it.

2. Clarify requirements

Functional

For an enterprise assistant/agent platform:

- upload/synchronize documents;
- ask questions and stream cited answers;
- track conversations;
- execute approved tools;
- inspect run/job status;
- cancel work;
- submit feedback;
- view audit/escalation history.

Non-functional

Clarify:

- p95 latency/time to first token;
- availability and durability;
- requests and concurrent generations/workflows;
- tenant isolation and data classification;
- freshness and deletion/ACL propagation;
- audit/compliance/retention;

- geography/data residency;
- cost per request/tenant;
- quality and abstention targets;
- recovery objectives.

State MVP and deferred scope to prevent design sprawl.

3. Capacity

Only estimate what changes architecture.

```
average QPS = requests/day ÷ 86,400
peak QPS = average QPS × peak multiplier
concurrency = QPS × average duration
storage growth = objects × size + chunks/vectors + metadata + logs
bandwidth = QPS × payload bytes
```

For AI include:

- input/output tokens;
- embedding volume;
- vector dimensions/metadata;
- active streams/workflows;
- tool calls per run;
- provider quotas;
- background ingestion throughput.

QPS alone is insufficient: a 90-second workflow or streamed generation creates much higher concurrency than a short control-plane request.

State assumptions, calculate orders of magnitude, then derive decisions.

Worked source-note example:

```
50,000 daily active users × 4 runs/day
= 200,000 runs/day

average QPS
= 200,000 / 86,400
≈ 2.3 starts/second
```

If business-hour traffic peaks at `10×`, design the entry path for roughly `23` starts/second. If a workflow lasts `60` seconds:

```
active workflows ≈ 23 × 60 ≈ 1,380
```

That result drives queue/worker concurrency, provider quotas, tenant fairness, checkpoint storage, and admission control. Apply the same method to ingestion bytes, tokens, vector storage, and bandwidth, then include indexes, replication, logs, backups, and versioning overhead rather than treating logical payload size as physical storage.

4. API contracts

```
POST /v1/conversations
POST /v1/conversations/{id}/messages
POST /v1/documents/upload-url
POST /v1/documents/{id}/ingestion
GET  /v1/jobs/{id}
POST /v1/projects/{id}/runs
GET  /v1/runs/{id}
POST /v1/runs/{id}/cancellation
POST /v1/runs/{id}/approval
POST /v1/feedback
```

Discuss:

- authentication/authorization;
- idempotency;
- cursor pagination;
- synchronous versus `202` asynchronous;
- timeouts/cancellation;
- streaming events;
- stable errors;
- API/component versions.

5. Data model and access patterns

Core entities:

```
Tenant, User, Project
Conversation, Message
Document, Chunk, IngestionJob
AgentRun, ToolExecution, Approval
Artifact, Feedback, AuditLog, OutboxEvent
```

Every tenant-owned path carries trusted `tenant_id`.

Indexes come from queries:

```
last project runs:
(tenant_id, project_id, created_at DESC, run_id DESC)

idempotent creation:
(tenant_id, idempotency_key) UNIQUE

audit history:
(tenant_id, created_at DESC, audit_id DESC)
```

Storage:

- relational for transactional state and audit;
- object storage for documents/large artifacts;
- vector/keyword search for evidence;
- Redis for transient/recomputable acceleration;

- registry/MLflow for model/app lifecycle.

6. Low-level design deep dive

When asked to deep-dive a component, move from boxes to responsibilities, contracts, sequence, state, concurrency, failure, and tests.

Responsibility, cohesion, and coupling

- One class should have one clear reason to exist.
- High cohesion keeps related behavior together.
- Low coupling makes business logic depend on capabilities rather than vendors.

Instead of one `AgentManager` that validates, writes SQL, calls models, sends notifications, stores files, and calculates billing, separate:

```
RunService
RunRepository
WorkflowDispatcher
ToolGateway
ArtifactStore
AuditWriter
```

Inject interfaces such as `RunRepository` and `WorkflowDispatcher`; possible implementations can use different databases, queues, or workflow engines.

Reusable LLD sequence

1. List the important classes.
2. Give one responsibility per class.
3. Define important methods, inputs, return values, and errors.
4. Show the runtime call sequence.
5. Explain transactions, optimistic concurrency, idempotency, and duplicate messages.
6. Cover invalid input, authorization, dual-write failure, timeouts, cancellation races, and duplicate callbacks.
7. Include unit, repository integration, contract, concurrency, failure-injection, and end-to-end tests.

7. High-level enterprise architecture

```
Web / SDK clients
  ↓
WAF / gateway / load balancer
  ↓
Identity, tenant, rate/admission control
  ↓
Conversation / AgentRun API
  ├─ relational DB + outbox + audit
  ├─ Redis cache/limits
  ├─ AI orchestration
  │    ├─ retrieval service
  │    ├─ model gateway
  │    ├─ tool/policy gateway
  │    └─ approval/checkpoint
  └─ queue or durable workflow engine
       ↓
     workers

Documents → object storage → ingestion workers
          → parse/chunk/embed → vector + keyword index

Logs + metrics + traces + evaluations
```

Separate the interactive request path from ingestion, evaluation, and long-running workflows.

8. Enterprise document assistant flow

Ingestion

```
connectors
→ change detection and ACL extraction
→ raw storage/catalog
→ parse/quality checks
→ structure-aware chunks
→ metadata/permissions/version
→ embeddings + keyword index
→ golden evaluation
→ publish index
```

Store connector cursor, checksum, source version, ACL version, last indexed timestamp, and deletion tombstone.

Query

```
SSO identity/groups
→ authorization scope
→ hybrid retrieval with mandatory ACL filters
→ rerank
→ current authoritative evidence
→ grounded generation
→ citation/schema/safety validation
→ stream
→ trace/feedback
```

Authorization correctness is more important than recall. Unauthorized evidence must never reach the prompt.

9. AgentRun execution

Start

One transaction writes:

```
AgentRun
OutboxEvent
AuditLog
```

Return `202`. The outbox publisher dispatches work. Consumers assume at-least-once delivery.

State

```
QUEUED → RUNNING
RUNNING → WAITING_FOR_TOOL | WAITING_APPROVAL
WAITING_* → RUNNING
RUNNING → SUCCEEDED | FAILED | CANCEL_REQUESTED
CANCEL_REQUESTED → CANCELLED
```

Use optimistic concurrency and immutable terminal states.

Tool boundary

- allowlist;
- schema and business validation;
- tenant-scoped credentials;
- policy decision;
- approval for writes;
- idempotency key;
- timeout and reconciliation;
- immutable audit.

The model proposes; policy authorizes; business service executes.

10. Reliability deep dive

Failure	Protection

API commits but workflow not dispatched	Transactional outbox, retry, stale-run alert, reconciliation.
Worker crashes	Durable queue/checkpoint, idempotent resume.
Duplicate message/callback	Event ID, unique constraint, state validation.
Poison job	Max attempts, backoff, DLQ, replay tooling.
Timed-out external write	Persist unknown state and reconcile before retry.
Model outage	Timeout, circuit breaker, approved fallback or controlled error.
Vector store outage	Fail closed/cached approved result or controlled degradation.
Reranker outage	First-stage rank when acceptable.
Tenant overload	Quotas, fair queues, concurrency pools, admission control.
Bad release	Canary, infrastructure + behavioral gates, rollback.
Stale/unauthorized document	Freshness/ACL SLO, reconciliation, deletion propagation.

Queue plus custom workers is simple but requires custom state, timers, recovery, and versioning. A durable workflow engine fits long-running branches, retries, approval, and resumability at the cost of a new operational model and coupling.

11. Security and governance

- Authenticate every entry.
- Derive tenant from trusted identity, not request input.
- Enforce authorization on every DB/search/tool operation.
- Use least-privilege workload/tool credentials.
- Encrypt in transit/at rest.
- Store secrets in a manager.
- Redact/safely sample logs.
- Separate trusted instructions from untrusted content.
- Restrict tool egress and commands/URLs.
- Require clear approval for consequential actions.
- Audit privileged operations.
- Version/approve data, models, prompts, indexes, tools, graphs, and policies.
- Define retention/deletion and incident response.

12. Evaluation and observability

Telemetry layers

Infrastructure:

- resources, queues, restarts, DB/vector/cache latency.

Application:

- rate, errors, p95/p99, retries, timeouts, cache, admission.

AI quality:

- retrieval recall/precision;
- citations/groundedness;
- answer relevance/abstention;
- tool success/trajectory;
- safety/policy;
- tokens/cost.

Business:

- task resolution;
- escalation;
- successful workflows;
- user satisfaction/adoption;
- cost per resolved task.

A deployment is healthy only when infrastructure and AI behavioral metrics are healthy.

13. Cost and performance

Measure each stage. Optimize in order:

1. Remove unnecessary model calls.
2. Improve retrieval filters and context quality.
3. Deduplicate context.
4. Route simple work to smaller models.
5. Bound output and tool steps.
6. Batch offline work.
7. Cache safe versioned work.
8. Parallelize independent I/O.
9. Right-size compute and autoscale with useful signals.

Do not trade away retrieval or security quality blindly to lower token cost.

14. Architecture trade-offs

Decision	Option A	Option B	Framing
Modular monolith/ microservices	Simple operations	Independent scale/ ownership	Split on evidence.
Sync/async	Immediate/simple	Durable/buffered	Keep short user path sync; long work async.
Shared/dedicated tenant store	Efficient	Strong isolation	Offer tiers by risk.
Vector/hybrid	Semantic	Semantic + exact	Use measured enterprise identifier needs.

One model/gateway	Simple	Routing/fallback/ governance	Abstract when switching/policy is real.
Free-form agent/ graph	Flexible	Predictable	Bound autonomy around side effects.
Queue/durable engine	Low platform overhead	Recovery/state semantics	Match workflow duration/complexity.
Rolling/blue-green	Efficient	Fast reversal	Match release and compatibility risk.
Managed/self-hosted	Low ops	Control/locality	Compare total cost and governance.

Staff-level formula:

```
constraint
→ decision
→ benefit
→ trade-off
→ mitigation
→ metric
→ trigger to revisit
```

15. Role and leadership expectations

The IJP role notes emphasize:

- owning problem-to-production lifecycle;
- converting vague business needs into AI solutions;
- architecture decisions and trade-offs;
- data, deployment, monitoring, and governance;
- stakeholder communication;
- mentoring and guidance.

The two role emphases differ:

- The AI/Data Scientist direction emphasizes ML/NLP, foundation models, solution architecture, enterprise AI, and Watson/watsonx awareness.
- The Advanced Analytics/GenAI/Databricks direction emphasizes Databricks, Delta Lake, ETL/ELT, MLflow, RAG/advanced RAG, vector databases, and orchestration frameworks.

At a high level, connect IBM's Watson/watsonx and Granite awareness to trusted enterprise AI, model development/deployment, lifecycle management, governance, and hybrid-cloud needs. The interview goal is architectural understanding, not memorizing every product feature.

Project story:

```
problem and users
→ personal scope/ownership
→ constraints
→ architecture
→ two hard decisions
→ failure and systemic fix
→ technical/business impact
→ learning/evolution
```

Be precise about what you owned and influenced. Use measured impact when available; do not invent numbers.

If no real production incident is documented, do not manufacture one to complete the story template. Use an evidenced challenge, design risk, or lesson learned, explain the preventive/systemic response, and state clearly when a future improvement is hypothetical.

For ambiguity, state assumptions, identify the decision that most changes the architecture, and separate the reversible MVP choice from the production evolution trigger. For speed versus quality, protect non-negotiable security, data integrity, and rollback while staging lower-risk capabilities.

16. Mock design prompts

Multi-tenant RAG SaaS

Cover:

- tenant/ACL enforcement;
- ingestion/freshness/deletion;
- hybrid retrieval/citations;
- token/latency/cost;
- shared versus dedicated index;
- quality gates and feedback.

Agent platform with tools

Cover:

- tool registry/schema/version/risk;
- state/checkpoints/approval;
- idempotency and unknown outcomes;
- policy and audit;
- step/cost/time limits;
- fair tenancy and provider quotas.

Enterprise document assistant

Cover:

- connector sync/cursor;
- authoritative sources;
- exact identifiers and hybrid retrieval;
- conflicting versions;
- prompt injection in documents;

- citation/source UX.

Model-serving platform

Cover:

- batch versus online;
- model registry/alias;
- API versus dedicated serving;
- concurrency/batching;
- canary/champion/rollback;
- drift and business monitoring.

Project-grounded Staff-level synthesis: DPDK automation to BenchOps Copilot

Project scenario and architectural evolution

The two projects form one evidence-backed evolution:

```
manual, fragmented AMD DPDK benchmarking
→ deterministic automation platform
   setup + BIOS + templates + execution + statistics + parsing + reporting
→ truth-bearing operational data and reusable domain knowledge
→ DPDK BenchOps Copilot
   RAG + controlled workflow + deterministic tools + evaluation + deployment
safeguards
```

The first project supported seven networking benchmarks; Radheshyam personally designed and implemented DPDK crypto, DPDK vhost, and DPDK testpmd end to end. It supported multi-server execution, 10–50+ scenario campaigns, several OS/compiler combinations, parameterized BIOS and benchmark configuration, structured metrics, and run comparisons. The second project used those assets as the factual and operational substrate for grounded Q&A, plan assistance, and regression analysis.

How to apply the system-design sequence

1. Requirements and constraints.

- Users: performance and benchmark engineers, including people without deep DPDK expertise.
- Functional needs: configure/run benchmark campaigns, collect and compare results, ask tuning/regression questions, receive cited evidence, and generate safe structured commands/plans.
- Critical constraints: reproducibility across platform variations; no hallucinated commands or tuning facts; human control over BIOS/reboot-affecting actions; auditable AI/tool behavior.
- Evidenced outcome goals: reduce manual error and setup effort, make large campaigns practical, improve benchmark analysis, and scale knowledge beyond specialists.

2. Data and architecture.

```
parameter-driven UI
→ deterministic benchmark automation
   Ansible + BIOS automation + Xena + command templates
→ raw logs/statistics
→ workload-specific parsers
→ structured database + dashboards/comparisons

documents/logs/DB records/run artifacts
→ LlamaIndex normalization, phase chunks, metadata, vector index
→ LangGraph/LangChain workflow
→ MCP: RunQuery | LogFetch | RunDiff | CommandBuilder
→ verified cited answer or controlled plan

authoritative records/artifacts: Postgres + S3/MinIO
semantic retrieval: vector database
service/deployment: FastAPI + Kubernetes + Helm + HPA + Jenkins
```

3. Hard decisions and trade-offs.

Constraint	Decision	Benefit	Trade-off / mitigation
Many benchmark/ platform variations	Shared roles, modules, templates, and parameter-driven configuration	Repeatability and reuse	More validation and conditional behavior; preserve workload-specific parsers where formats differ.
Factual guidance must be explainable	RAG with benchmark-aware chunks, metadata, verification, and citations	Current/private evidence remains traceable	More ingestion/ retrieval latency and versioning work; use evaluation gates.
Commands and comparisons must be correct	Deterministic MCP tools and allowlisted templates	Reproducible, auditable operations	Less free-form flexibility; experts extend reviewed templates.
BIOS/reboot changes are disruptive	Human-controlled gate	Limits high-impact mistakes	Adds delay; apply approval only to the high-risk path.
Semantic search is not authoritative storage	Postgres/S3/MinIO for truth, vector database for discovery	Clear data ownership and recoverability	Synchronization and lineage obligations.
AI changes can regress silently	Golden-set CI gates plus canary/rollback thinking	Higher release confidence	Evaluation maintenance and longer delivery path.

Outcomes and evidence discipline

Documented outcomes include:

- the automation framework became the team's default DPDK/networking campaign path;

- 10–50+ scenarios and multi-server runs became practical;
- environment, BIOS, execution, collection, parsing, and reporting were combined into a repeatable pipeline;
- reusable documentation, roles, scripts, modules, and templates supported onboarding and extension;
- the Copilot returned grounded cited assistance, reduced reliance on tribal knowledge, kept operational capabilities deterministic, and improved release confidence through evaluation gates.

Do not invent exact percentages for time saved, error reduction, retrieval accuracy, latency, availability, cost, or adoption. If an interviewer asks for numbers, give only measurements you can substantiate or say what you would measure.

How to present it in a Senior interview

Use a 90-second core story:

> "I led an AMD-centric DPDK automation platform because manual multi-configuration campaigns were slow and difficult to reproduce. I started with DPDK crypto, generalized proven setup and execution patterns into reusable Ansible roles, Python BIOS/Xena modules, command templates, parsers, and a database-backed comparison flow, then extended the platform across seven benchmarks. Once that deterministic foundation and domain knowledge existed, I designed a BenchOps Copilot. LlamaIndex organized the benchmark knowledge, LangGraph controlled the workflow, and narrow MCP tools handled run lookup, log fetch, comparison, and allowlisted command building. The key decision was that AI could synthesize and propose, but deterministic services retained truth and execution. BIOS/reboot actions remained human-controlled, and CI gated groundedness, retrieval, citations, tool reliability, and latency. The result was repeatable large campaigns and grounded assistance without weakening operational safety."

Then deep-dive one component you personally owned: Xena integration, BIOS automation, a parser/comparison flow, benchmark-aware ingestion, or the MCP/verification path. Explain concrete inputs, outputs, failures, and tests rather than listing the whole stack again.

How to present it in a Staff interview

Lead with the system-level insight:

> "The AI system was valuable because we first created a deterministic operational substrate. I treated structured runs, parsers, templates, and documentation as platform capabilities, then allocated probabilistic behavior only to interpretation and synthesis."

Use the Staff formula:

```
constraint: hallucinated commands and platform-mismatched advice were
unacceptable
→ decision: separate RAG reasoning from deterministic execution
→ benefit: grounded assistance with reproducible tools
→ trade-off: more components, versioning, evaluation, and latency
→ mitigation: narrow tools, metadata, verification, audit, approval, CI
gates
→ metric: retrieval quality, groundedness, citation coverage, tool success/
error, p95 latency
→ revisit trigger: measured retrieval gaps, new benchmark families, or
operational bottlenecks
```

Also show organizational leverage: you led a three-person team in the original platform, drove domain learning and AMD-centric documentation, collaborated with UI/reporting stakeholders, and turned reusable operational knowledge into an AI-assisted capability. That is stronger Staff evidence than presenting framework selection alone.

Evidence boundaries and hypothetical evolution

The project files do not substantiate a particular cloud provider, Terraform, Databricks/Delta/MLflow, multi-agent execution, an event broker/outbox, checkpoint persistence, tenant-isolation implementation, a frontend framework, exact capacity/SLO figures, or a specific production incident and its measured remediation.

If asked how you would evolve the design, label proposals explicitly:

- **Hypothetical:** add a durable workflow/queue and idempotent state model if runs must survive process failure at larger scale.
- **Hypothetical:** formalize infrastructure with Terraform if repeatable multi-environment provisioning becomes an ownership requirement.
- **Hypothetical:** add stronger tenant/identity boundaries before offering the internal platform as multi-tenant software.
- **Hypothetical:** evaluate a governed lakehouse/MLflow lifecycle only if benchmark and evaluation lineage outgrow the existing storage/release model.

17. High-signal interview questions

1. How do you begin a GenAI system-design interview?
2. Why calculate concurrency as well as QPS?
3. What belongs in synchronous versus asynchronous paths?
4. How does the outbox prevent a lost run?
5. Why assume at-least-once delivery?
6. How do you cancel a long-running workflow?
7. SQL versus NoSQL versus cache versus object/vector store?
8. Queue workers versus durable workflow engine?
9. How do you prevent one tenant from consuming all capacity?
10. How do you measure RAG separately from generation?
11. How do you protect against prompt injection and cross-tenant retrieval?
12. How do you deploy a new model/prompt/index safely?

13. What is the largest production risk in an enterprise assistant?
14. How do you present a failure and lesson?
15. What makes an answer Staff-level rather than a component list?
16. How do you move from HLD to an LLD for `RunService`?
17. How do the two IBM IJP role emphases differ?
18. How do you tell a credible project story when no measured incident or percentage improvement is documented?

18. Five-to-seven-minute answer skeleton

1. State scope and deferred features.
2. Give measurable NFR assumptions.
3. Estimate QPS, concurrency, storage, and tokens.
4. Define critical APIs.
5. Define entities and access-driven indexes.
6. Draw sync request and async ingestion/workflow paths.
7. Deep-dive one hard problem: outbox, retrieval, tools, or tenancy.
8. Cover failures, security, observability, evaluation, and cost.
9. State trade-offs, bottlenecks, and next evolution trigger.

19. Final checklist

- [ ] Clarify users, functions, NFRs, and MVP.
- [ ] Estimate QPS, concurrency, storage, bandwidth, tokens, and background throughput.
- [ ] Define stable APIs, entities, indexes, and storage roles.
- [ ] Draw interactive, ingestion, agent, and deployment paths.
- [ ] Explain outbox, at-least-once, state machines, retries, and reconciliation.
- [ ] Enforce tenant, ACL, tool, secret, and audit boundaries.
- [ ] Measure infrastructure, application, AI quality, and business.
- [ ] State cost/performance choices.
- [ ] Use constraint-to-evolution trade-off language.
- [ ] Deliver both 90-second and seven-minute versions.
- [ ] Deep-dive responsibilities, interfaces, sequence, concurrency, edge cases, and tests.
- [ ] Separate implemented facts, transferable design interpretation, and hypothetical evolution.

Source notes

- System Design Interview Coaching
- IBM IJP Roles Preparation
- Enterprise GenAI Solution Design

- Cloud AI Architecture
- Deploying ML Models API
- MLOps for AI Systems
- Vanilla RAG
- Vanilla RAG and Frameworks
- Capstone Revision Day 3
- DPDK Automation for Network Packet Processing
- DPDK BenchOps Copilot